

BACHELOR OF TECHNOLOGY
FINAL YEAR PROJECT REPORT

Android Dynamic Malware Analysis Framework

Runtime Behavioral Profiling · Frida Instrumentation · ML Classification

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Technology in Computer Science & Engineering

Academic Year 2024–2025

Department of Computer Science & Engineering

Abstract

Android malware has evolved significantly beyond what signature-based or permission-based detection systems can reliably intercept. This project presents a complete dynamic malware analysis framework that observes application behavior at runtime, inside a controlled Android emulator, with behavioral hooks injected directly into the application process using the Frida dynamic instrumentation toolkit.

The framework captures runtime behavioral events — API calls, file accesses, network endpoints, device identifier reads, and dynamic code execution — without prior knowledge of what signals to look for. A behavioral quality scoring and filtering pipeline was developed that addresses the critical problem of noisy dynamic analysis datasets. The core empirical finding demonstrates that a quality-filtered 600-sample dataset achieves ~96% classification accuracy, outperforming a larger 1,044-sample noisy dataset (~88–91%), validating that behavioral signal quality dominates raw sample count.

Final Accuracy	Dataset Size	Models Evaluated	Pipeline Stages
~96% (Dataset E)	600 (quality-filtered)	10+ classifiers	7 end-to-end stages

Keywords: Android Malware Detection, Dynamic Analysis, Frida Instrumentation, Behavioral Feature Engineering, Machine Learning, Quality-Aware Dataset Curation.

Table of Contents

Abstract	2
Table of Contents	3
1. Introduction and Research Motivation	5
1.1 Background	5
1.2 Problem Statement — Why Static Analysis Fails	5
1.3 Research Objectives	5
2. System Architecture	7
2.1 Overview	7
2.2 Technology Stack.....	7
3. Frida Instrumentation and Behavioral Hooking.....	9
3.1 Instrumentation Architecture.....	9
3.2 Hooks Implemented.....	9
3.3 Behavioral JSON Trace Format.....	9
4. Dataset Evolution — The Research Journey.....	11
4.1 Key Finding — Quality vs. Quantity	11
5. Behavioral Quality Filtering Methodology	12
5.1 Scoring Dimensions.....	12
5.2 Weighted Event Signal Scores	12
5.3 Composite Quality Score Formula	12
6. Feature Engineering Pipeline	14
6.1 Feature Categories.....	14
6.2 Behavioral Co-occurrence Interaction Features.....	14
7. Machine Learning Pipeline	16
7.1 Experimental Setup	16
7.2 Models Trained.....	16
8. Experimental Results	17
8.1 Dataset Comparison — Accuracy Across Phases	17
8.2 Full Model Comparison — Dataset E.....	17
8.3 Top Feature Importance (RFE Ranking).....	17
9. Engineering Challenges and Limitations	19
9.1 Key Engineering Challenges Encountered	19
9.2 System Limitations	19
10. Future Work	21
11. Conclusion	22
References	23

1. Introduction and Research Motivation

1.1 Background

Android remains the world's most widely deployed mobile operating system, making it the primary target for mobile malware authors. Modern adversarial Android applications employ multi-stage obfuscation, dynamic code loading, reflection-based API invocation, encrypted payloads, and time-delayed activation logic — all specifically designed to appear benign under static inspection.

This project builds a complete dynamic malware analysis framework that sidesteps these evasion strategies entirely. Rather than inspecting what an APK looks like, the system observes what it does — at runtime, inside a controlled Android emulator, with behavioral hooks injected directly into the application process.

1.2 Problem Statement — Why Static Analysis Fails

Traditional Android security tools operate on the static artifact — the decompiled APK, its manifest file, its declared permissions, and its code graph. These approaches carry well-understood failure modes when confronting modern malware:

Evasion Technique	Effect on Static Analysis
Code obfuscation (ProGuard, DexGuard)	API call signatures become unrecognizable
Reflection-based invocation	Method calls are invisible to static call graphs
Dynamic DEX loading	Malicious code is absent from the initial APK
Encrypted payload delivery	Malicious logic is hidden until runtime
Native code (JNI)	Bypasses Java-layer static inspection entirely
Time-delayed / condition-gated activation	Malicious branches never triggered in analysis
Polymorphic malware families	Signature databases perpetually out-of-date

1.3 Research Objectives

The core methodological commitment of this framework is: capture behavior first, engineer features second, and let runtime execution reveal the truth that static analysis cannot. The specific objectives are:

1. Design and implement a Frida-based runtime instrumentation framework for Android APKs.
2. Build an end-to-end pipeline from raw APK ingestion through ML-ready feature generation.
3. Investigate the relationship between behavioral data quality and classifier performance.
4. Develop a behavioral quality scoring and filtering algorithm for dynamic analysis datasets.
5. Evaluate multiple machine learning classifiers and ensemble strategies on the resulting data.

2. System Architecture

2.1 Overview

The framework operates as a seven-stage end-to-end pipeline. Each stage is independently modular, enabling future improvements to any stage to propagate naturally to improved classification performance.

DYNAMIC ANALYSIS PIPELINE — END-TO-END FLOW	
Stage 1 APK Input	— Raw malware and benign APKs placed in categorized input directories
Stage 2 Android Emulator	— APK installation and execution inside sandboxed x86_64 emulator (API 30)
Stage 3 Frida Instrumentation	— JavaScript hooks injected into application process; behavioral events captured in real time
Stage 4 Behavioral JSON Traces	— Per-APK structured event logs written to dataset/ directory
Stage 5 Quality Filtering	— All traces scored and ranked; top-300 per class retained as Dataset E
Stage 6 Feature Engineering	— Statistical, behavioral, network, and co-occurrence features extracted to CSV
Stage 7 ML Classification	— 10+ classifiers trained, tuned, and ensembled; best model serialized

2.2 Technology Stack

Layer	Tool / Library	Role
Execution Environment	Android Studio Emulator	Sandboxed APK execution (API 30, x86_64)
Device Communication	ADB (Android Debug Bridge)	Emulator control, APK installation and management
APK Inspection	AAPT	Package name extraction and manifest parsing
Runtime Instrumentation	Frida + frida-tools	Dynamic hooking and API call interception
Pipeline Automation	Python 3.10+	End-to-end pipeline orchestration
Data Storage	JSON	Structured behavioral trace storage per APK
Data Processing	Pandas, NumPy	Feature engineering and dataframe operations
ML Algorithms	Scikit-learn, XGBoost, LightGBM, CatBoost	Classification model training and evaluation

Imbalance Handling	imbalanced-learn (SMOTE)	Synthetic minority oversampling
Deduplication	datasketch (MinHash LSH)	Near-duplicate behavioral trace detection

3. Frida Instrumentation and Behavioral Hooking

3.1 Instrumentation Architecture

The script.js file contains Frida JavaScript hooks that intercept Android API calls at the process level. Hooks are injected before any application code executes using the `frida.spawn()` + `session.resume()` pattern, ensuring complete behavioral coverage from process start.

Each hook emits a structured JSON event to the Frida message channel, which is received by `runner.py` and accumulated into a per-APK behavioral trace. The Android Monkey event generator drives UI interaction to stimulate behavioral branches.

3.2 Hooks Implemented

Category	Hook Target	Captured Information
Network	<code>java.net.URL.openConnection()</code>	URL, connection target hostname
Network	<code>java.net.HttpURLConnection</code>	HTTP method, host
Network	<code>android.webkit.WebView.loadUrl()</code>	WebView-loaded URLs
Network	<code>java.net.Socket</code>	Socket host and port
File System	<code>java.io.File.<init>()</code>	File path on construction
File System	<code>java.io.FileOutputStream.<init>()</code>	Write target paths
Device Info	<code>Settings.Secure.getString()</code>	android_id, device identifiers
Device Info	<code>TelephonyManager.getDeviceId()</code>	IMEI / device ID access
Device Info	<code>TelephonyManager.getSubscriberId()</code>	IMSI access
Runtime Exec	<code>java.lang.Runtime.exec()</code>	Executed shell command strings
Runtime Exec	<code>ProcessBuilder.start()</code>	Process command arrays
Dynamic Code	<code>DexClassLoader.<init>()</code>	DEX file paths being loaded dynamically
Cryptography	<code>Cipher.getInstance()</code>	Cipher algorithm requested
Cryptography	<code>Cipher.doFinal()</code>	Cryptographic operation triggered
SMS	<code>SmsManager.sendTextMessage()</code>	Destination number, message body

3.3 Behavioral JSON Trace Format

Each successfully processed APK produces a structured JSON trace. A representative malware trace exhibits clusters of high-signal events:

Event Type	Field	Example Value
network	url	<code>http://c2-server.ru/gate.php?id=abc123</code>

crypto	algorithm	AES/CBC/PKCS5Padding
device_id	key	android_id
runtime_exec	cmd	su -c id
dex_load	path	/data/data/com.evilmalware.app/files/payload.dex
socket	host:port	185.220.101.47:4444
sms	destination	+1XXXXXXXXXX (premium number)

4. Dataset Evolution — The Research Journey

The research progressed through three distinct dataset phases. Each phase produced a materially different experimental outcome, and the transitions between phases represent the core research insights of this project.

Attribute	Phase 1	Phase 2	Phase 3 (Dataset E)
Total Samples	~600	1,044	600
Malware Samples	~300	~522	300 (filtered)
Benign Samples	~300	~522	300 (filtered)
Dead Executions	Minimal	93 traces (≤ 2 events)	0 (removed)
Near-Duplicates	Low	259 traces (Jaccard ≥ 0.85)	0 (removed)
Startup-Only Traces	Minimal	118 traces	0 (removed)
Sparse Traces	Minimal	136 traces (< 15 events)	0 (removed)
Best Accuracy	~92%	~88–91%	~96%

4.1 Key Finding — Quality vs. Quantity

The counterintuitive result — a smaller dataset outperforming a larger one — is explained by the mechanics of noise in classifier training:

Mechanism	Explanation
Class Separability	Dead/sparse traces cluster near benign samples in feature space, shrinking decision boundary margins
Feature Consistency	Sparse traces produce degenerate zero-feature vectors, training classifiers to associate zeros with both classes
Information Redundancy	Near-duplicate traces add sample count without adding information — functionally equivalent to label noise
Model Generalization	A classifier trained on behaviorally rich, non-redundant samples generalizes better to unseen APKs

5. Behavioral Quality Filtering Methodology

The behavioral quality filtering pipeline is the most technically novel component of this project. Each trace is scored across five independent dimensions and ranked within each class. The top 300 per class are retained to form Dataset E.

5.1 Scoring Dimensions

Dimension	Metric	Description
1. Volume	$\log_{10}(n_events) / \log_{10}(max_events)$	Raw count of behavioral events; low count penalized below threshold of 15
2. Diversity	$n_distinct_types / total_possible_types$	Number of distinct behavioral event categories observed
3. Entropy	Shannon entropy over event distribution	Traces dominated by a single event type receive low entropy scores
4. Weighted Signal	$\sum (count_i \times weight_i)$	High-signal events (dex_load, runtime_exec, sms, socket) receive elevated weights
5. Duplicate Penalty	MinHash LSH (Jaccard ≥ 0.85)	Near-duplicate traces are penalized; only highest-scoring in a cluster is retained

5.2 Weighted Event Signal Scores

Event Type	Weight Rationale	Malware Association
runtime_exec	Shell command execution is rare in benign apps	Root access, command injection
dex_load	Dynamic DEX loading is a common evasion pattern	Payload delivery, code hiding
sms	SMS sending indicates monetization or spread	Banking trojans, premium SMS fraud
socket	Raw socket access indicates C2 communication	Remote access trojans, botnets
device_id	Device fingerprinting indicates data harvesting	Spyware, adware
crypto	Cryptography often associated with C2 encryption	Encrypted channels, ransomware
subscriber_id	IMSI access indicates mobile banking focus	Mobile banking malware

5.3 Composite Quality Score Formula

```
quality_score = w1 x volume_score + w2 x richness_score + w3 x  
entropy_score + w4 x weighted_signal_score - duplicate_penalty
```

6. Feature Engineering Pipeline

The `feature_engineering.py` script processes each JSON behavioral trace into a flat feature vector. Features are organized into five categories, with additional interaction terms engineered to capture compound behavioral patterns.

6.1 Feature Categories

Category	Feature	Description
Statistical	n_events	Total runtime events captured
Statistical	event_density	Events per second of execution
Statistical	ts_entropy	Shannon entropy of timestamp distribution
Behavioral Count	cnt_network	Network connection events
Behavioral Count	cnt_runtime_exec	Runtime.exec() / ProcessBuilder calls
Behavioral Count	cnt_dex_load	Dynamic DEX class loading events
Behavioral Count	cnt_crypto	Cryptographic API invocations
Behavioral Count	cnt_sms	SMS send attempts
Behavioral Count	cnt_device_id	Device identifier access events
Presence Flag	has_dex_load	Dynamic DEX loading occurred (binary)
Presence Flag	has_runtime_exec	Process execution attempted (binary)
Presence Flag	has_subscriber_id	IMSI access observed (binary)
Network	unique_domains	Count of unique network destinations
Network	suspicious_url_count	URLs matching known malicious patterns
Quality	richness_score	Behavioral category diversity score
Quality	behavioral_entropy	Shannon entropy over event type distribution

6.2 Behavioral Co-occurrence Interaction Features

Beyond raw counts, co-occurrence flags capture compound behavioral patterns that are more discriminative than individual features:

Interaction Feature	Components	Malware Pattern Encoded
crypto_x_network	has_crypto AND has_network	Encrypted C2 communication channel
deviceid_x_network	has_device_id AND has_network	Device fingerprint exfiltration
dex_x_exec	has_dex_load AND has_runtime_exec	Payload delivery and execution
webview_x_url	has_webview AND has_url	WebView-based phishing
sms_x_deviceid	has_sms AND has_device_id	Mobile banking trojan pattern
subscriberid_x_network	has_subscriber_id AND has_network	IMSI exfiltration pattern
high_signal_ratio	(exec+dex+sms+socket) / n_events	Proportion of malware-indicative events

7. Machine Learning Pipeline

7.1 Experimental Setup

Dataset E (600 samples, perfectly balanced 300:300) is split using stratified holdout. A RobustScaler is applied to normalize count features, which are typically heavy-tailed. 5-fold stratified cross-validation is used for hyperparameter tuning.

Training Set	Validation Set	Cross-Validation
480 samples (80%) 240 benign + 240 malware	120 samples (20%) 60 benign + 60 malware	StratifiedKFold 5 folds, random_state=42

7.2 Models Trained

Model	Key Configuration
Random Forest (baseline)	n_estimators=300, max_depth=20, class_weight='balanced'
XGBoost	n_estimators=300, learning_rate=0.1, max_depth=8
LightGBM	n_estimators=300, learning_rate=0.1, num_leaves=31
CatBoost	iterations=500, learning_rate=0.05, depth=8, early_stopping_rounds=50
MLP (Neural Network)	hidden_layers=(128,64,32), activation=relu, early_stopping=True
KNN	n_neighbors=5
RF + SMOTE	Random Forest with SMOTE oversampling on training split
Stacking Ensemble	RF + XGB + LGB as base learners; Logistic Regression meta-learner
Soft Voting Ensemble	Soft-vote RF + XGB + LGB; grid-searched optimal weights
Weighted Ensemble (tuned)	Grid search over $[1,2,3]^3 = 27$ weight combinations

8. Experimental Results

8.1 Dataset Comparison — Accuracy Across Phases

Model	Phase 1 (~600)	Phase 2 (1,044 raw)	Dataset E (600 filtered)
Random Forest	~91–92%	~88–91%	~93–94%
XGBoost	~90%	~87–89%	~94–95%
LightGBM	—	—	~94%
CatBoost	—	—	~95–96%
Stacking Ensemble	~88–89%	~86–88%	~92–93%
Voting Ensemble (tuned)	—	—	~95–96%
5-fold CV (RF)	—	—	95.7%

8.2 Full Model Comparison — Dataset E

Model	Accuracy	Precision	Recall	F1 Score	Notes
Tuned CatBoost	~96%	High	High	High	Best single model
Weighted Ensemble (Optimised)	~95–96%	High	High	High	Best ensemble
Ensemble (Acc-Tuned RF+XGB+LGB)	~95%	High	High	High	—
Tuned XGBoost	~94–95%	High	High	High	—
Tuned LightGBM	~94%	High	High	High	—
Tuned Random Forest	~93–94%	High	High	High	—
Ensemble (RFE Features)	~93%	High	High	High	Top-20 features
Stacking Ensemble	~92–93%	Good	Good	Good	—
RF + SMOTE	~92%	Good	Good	Good	Minimal SMOTE gain
Random Forest (baseline)	~91–92%	Good	Good	Good	Baseline
Neural Network (MLP)	~88–90%	Moderate	Moderate	Moderate	—
KNN	~85–87%	Moderate	Moderate	Moderate	Lowest accuracy

8.3 Top Feature Importance (RFE Ranking)

Recursive Feature Elimination consistently selected the following as top-ranked features across all classifiers, validating the feature engineering design:

Rank	Feature	Why It Matters
1	weighted_event_score	Aggregates high-signal behavioral events with malware-tuned weights
2	dex_x_exec	Co-occurrence of dynamic loading + execution — payload delivery pattern
3	high_signal_ratio	Proportion of events that are malware-indicative
4	crypto_x_network	Encrypted C2 communication pattern
5	richness_score	Behavioral diversity of the execution trace
6	cnt_dex_load	Raw count of dynamic DEX loading events
7	cnt_runtime_exec	Raw count of shell execution events
8	log_weighted_score	Log-transformed weighted score (skew-stabilized)
9	richness_x_volume	Interaction of diversity and total event volume
10	deviceid_x_network	Device fingerprint exfiltration pattern

9. Engineering Challenges and Limitations

9.1 Key Engineering Challenges Encountered

Challenge	Root Cause	Mitigation Applied
Frida version mismatch	Client-server protocol is version-locked; any mismatch causes silent failures	Maintained strict version pinning between frida-tools and server binary
ABI mismatch crashes	Some APKs compiled for ARM only; crash immediately on x86_64 emulator	Detected via UnsatisfiedLinkError in ADB logs; filtered during quality scoring
Emulator instability	Batch runs of 50+ APKs cause memory pressure and freezes	Restarted emulator every 20–30 APKs; ran in smaller batches
Dormant malware	Banking trojans / RATs activate only via C2 commands	Quality filtering penalizes low-signal traces; effectively removes dormant samples
Sparse behavioral traces	Applications require auth or complex UI flows before exhibiting behavior	Monkey event generator partially mitigates; identified as key limitation
Near-duplicate APKs	Multiple APKs from same family produce near-identical behavioral traces	MinHash LSH deduplication with Jaccard ≥ 0.85 threshold
Emulator detection by malware	Sophisticated samples detect emulator hardware fingerprints	Partial mitigation via realistic device properties; identified for future work
APK installation failures	SDK constraints, corrupted signatures, package conflicts	Automated cleanup; retry logic; filtered from dataset

9.2 System Limitations

Limitation	Impact and Scope
Behavioral coverage bound	Analysis quality is bounded by UI interaction depth; Monkey cannot simulate realistic user flows
Emulator detection evasion	Sophisticated malware suppresses behavior in emulated environments
Single execution window	Time-delayed or C2-dependent malware may not trigger within the fixed window
No PCAP network capture	Packet-level network details (payloads, TLS fingerprints) are not captured
Dataset scale	600-sample Dataset E is modest compared to production-scale datasets

Feature stationarity	Behavioral features may become less discriminative as malware techniques evolve
-----------------------------	---

10. Future Work

Future Direction	Description and Expected Impact
Structured UI Automation	Replace Monkey with UIAutomator2 / Appium for context-aware interaction — significantly improves behavioral coverage
Anti-Emulation Hardening	Implement IMEI spoofing, sensor simulation, and realistic device property injection to reduce emulator detection surface
Temporal Sequence Features	Add event-type n-gram TF-IDF features that capture ordering and sequential structure of behavioral events
Network Traffic Analysis	Integrate tcpdump / mitmproxy for raw PCAP capture alongside Frida behavioral events
Graph-based Representation	Model behavioral traces as directed API call graphs and apply Graph Neural Networks for classification
Multi-Run Averaging	Execute each APK across multiple independent runs with different Monkey seeds to reduce interaction variance
Online Learning	Deploy classifier with online learning wrapper to adapt to evolving malware distributions without full retraining

11. Conclusion

This project delivers a complete, research-grade framework for Android malware detection through dynamic behavioral analysis. The framework demonstrates the following findings:

1.	Frida-based runtime instrumentation produces behavioral features that are meaningfully discriminative for Android malware classification, achieving up to ~96% accuracy on a quality-filtered behavioral dataset.
2.	Dynamic analysis surface area is bounded by interaction quality. The most significant limitation is not the classifier architecture or feature set — it is the depth of behavioral exploration achievable through automated UI interaction.
3.	Behavioral data quality is the dominant variable in classification performance — more so than dataset size, model complexity, or feature engineering sophistication. The transition from a noisy 1,044-sample dataset (~88–91%) to a quality-filtered 600-sample dataset (~96%) is the central empirical finding.
4.	Quality-aware dataset curation is a first-class research contribution. The behavioral quality filtering pipeline is not a preprocessing convenience — it is a core methodological contribution that directly enables the final accuracy levels achieved.
5.	The framework is modular, reproducible, and extensible. Each pipeline stage — instrumentation, log parsing, quality scoring, feature engineering, model training — is independently replaceable.

The core research insight — that behavioral signal quality is a stronger predictor of classifier performance than raw dataset size — represents a reproducible, empirically grounded contribution to the field of dynamic Android malware analysis.

References

- Android Security Bulletin. Google, 2024. <https://source.android.com/docs/security/bulletin>
- Frida Dynamic Instrumentation Toolkit. Ole Andre Vadla Ravnas. <https://frida.re/>
- Arp, D. et al. (2014). DREBIN: Effective and Explainable Detection of Android Malware. NDSS Symposium.
- Enck, W. et al. (2014). TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. ACM TOCS.
- Cen, L. et al. (2015). Simulating a Global Passive Adversary for Attacking Tor-Like Anonymity Systems. ESORICS.
- Scikit-learn: Machine Learning in Python. Pedregosa et al. (2011). JMLR 12, pp. 2825–2830.
- Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. ACM KDD.
- Ke, G. et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS.
- Prokhorenkova, L. et al. (2018). CatBoost: Unbiased Boosting with Categorical Features. NeurIPS.
- Chawla, N. et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. JAIR 16.